PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

CONSTRUCCIÓN Y DOCUMENTACIÓN DE UNA API CON EXPRESS

Presentado a:	Yoly Alexandra Guevara
Por Aprendiz:	**Luis fernnado conjeo quiñones**
Ficha:	3312932

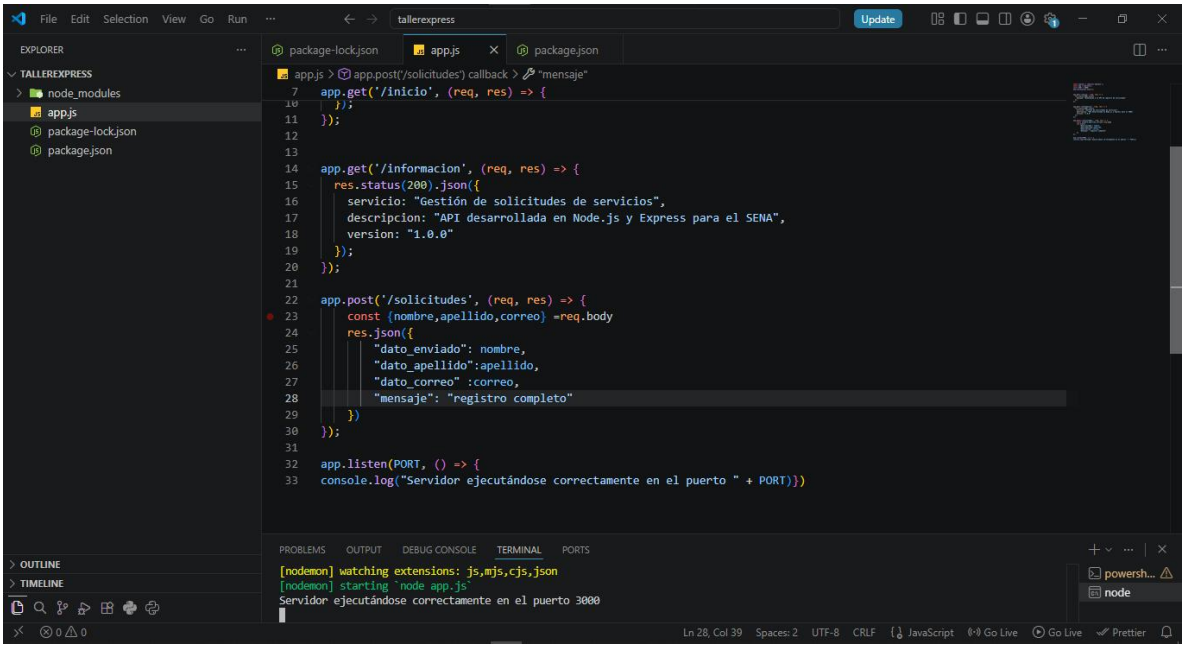
Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 20 del 10 del año 2026

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Punto 1:

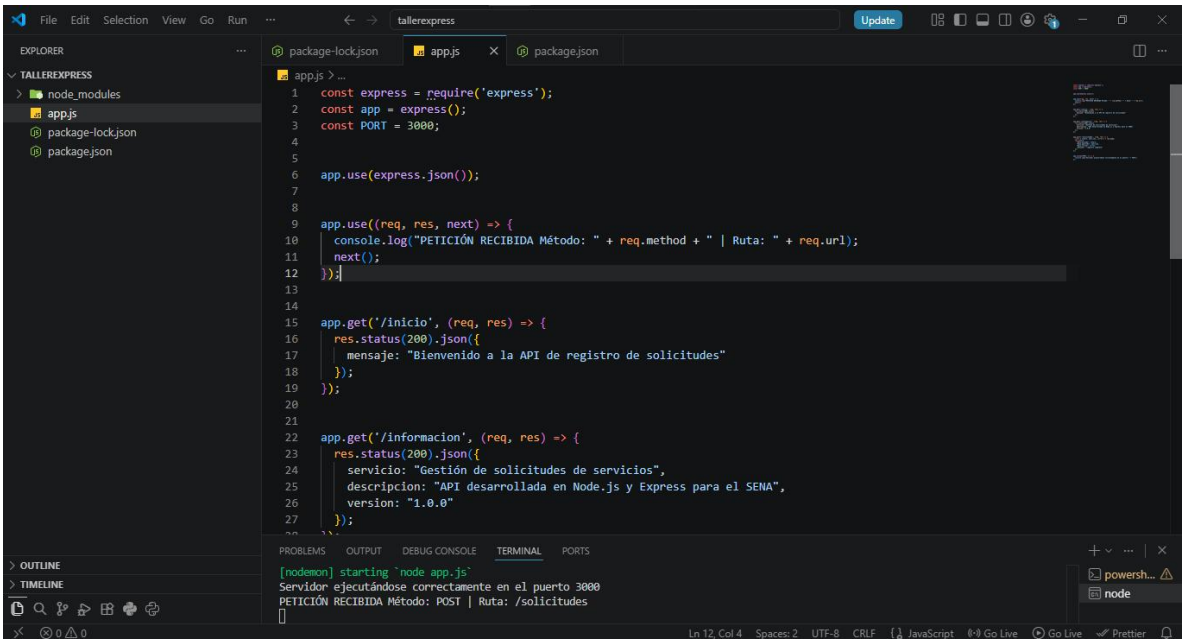
En este punto se estructuró la base de la API REST con **Node.js** y **Express**. Se habilitó el middleware express.json() para procesar datos en dicho formato, se crearon las rutas GET /inicio e /informacion para responder con la información del servicio y un código HTTP 200, y la ruta POST /solicitudes para capturar los datos enviadas desde el cuerpo de la petición (req.body) y confirmar el registro. Finalmente, el servidor queda escuchando activamente en el puerto 3000.

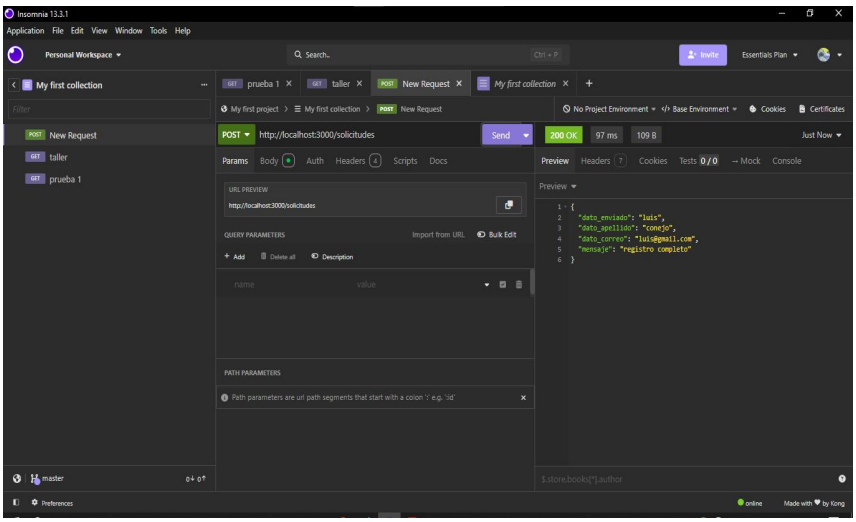


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Punto 2: Construcción del middleware propio

Para este punto creamose e implementamos un **middleware de registro** usando app.use(), colocándolo antes de las rutas. Su trabajo es interceptar cada petición que entra, extraer el método HTTP (req.method) y la ruta (req.url), e imprimirlos en la consola. Una vez registrado el mensaje, usamos next() para darle paso libre a la petición hacia la ruta correspondiente sin bloquear el servidor



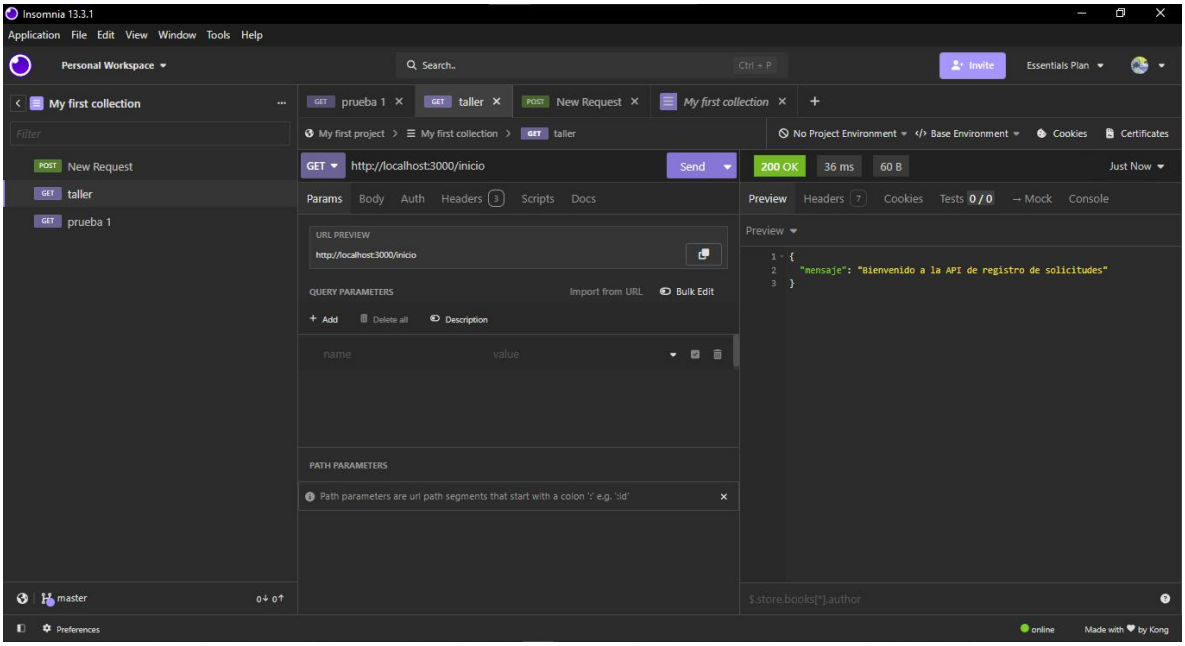


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

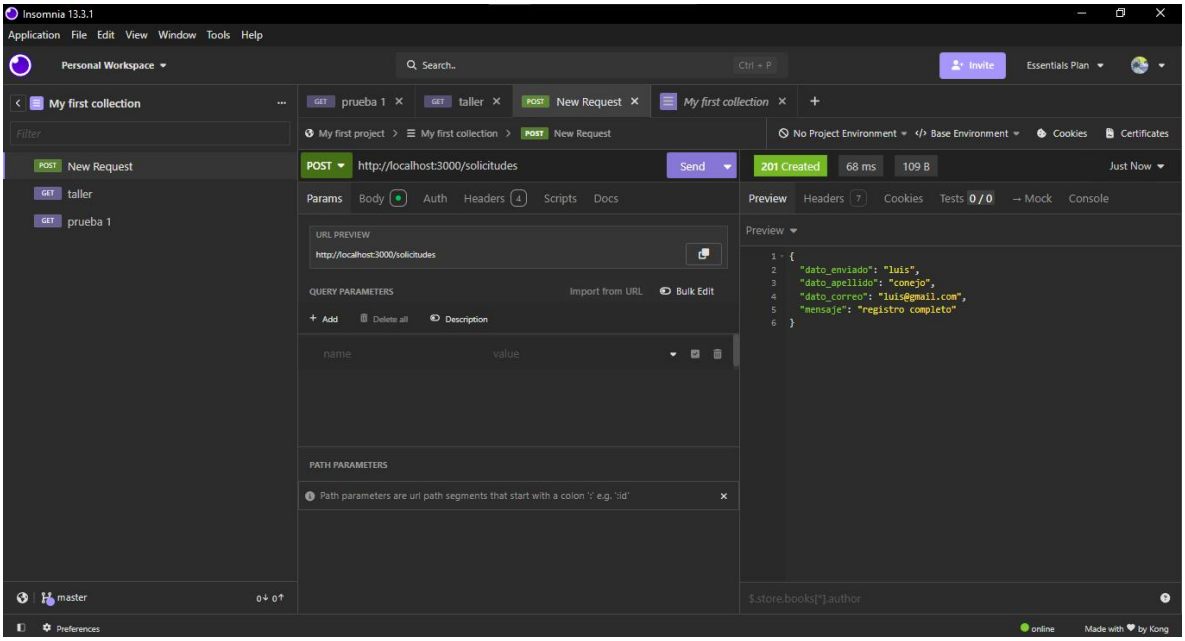
Punto 3:Códigos de estado HTTP y manejo de errores

Codigos:

200 OK: Indica éxito general en peticiones GET. Lo usamos para devolver la información de bienvenida e
institucional.

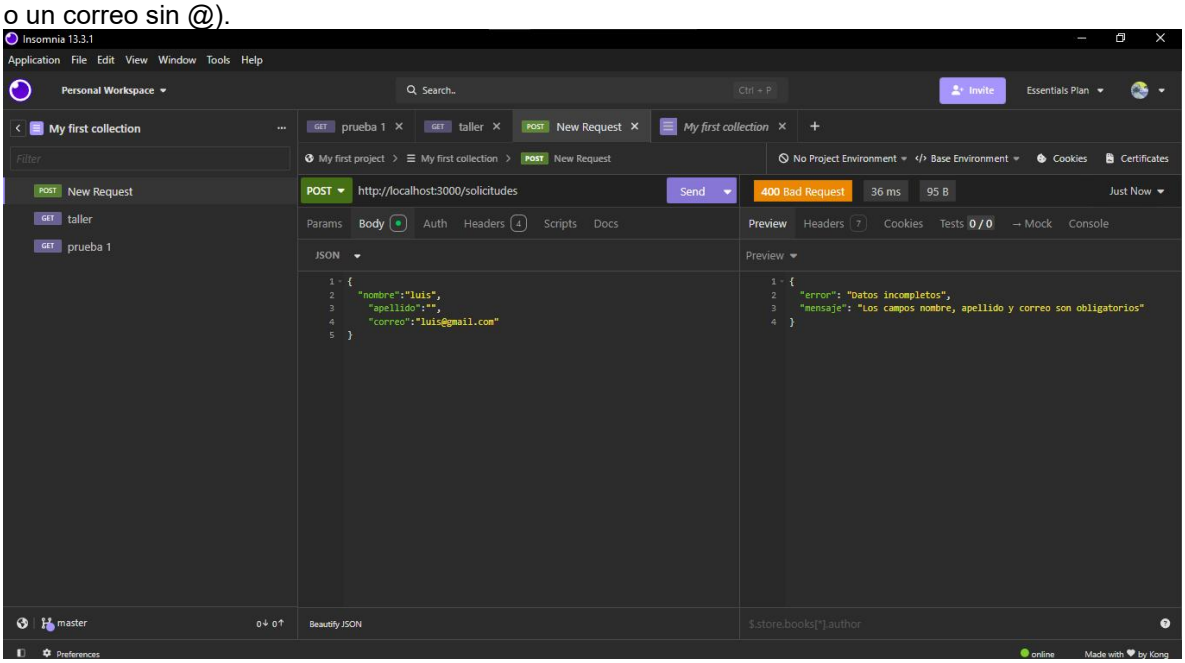


201 Created: Confirma que un recurso fue creado o procesado con éxito en peticiones POST.

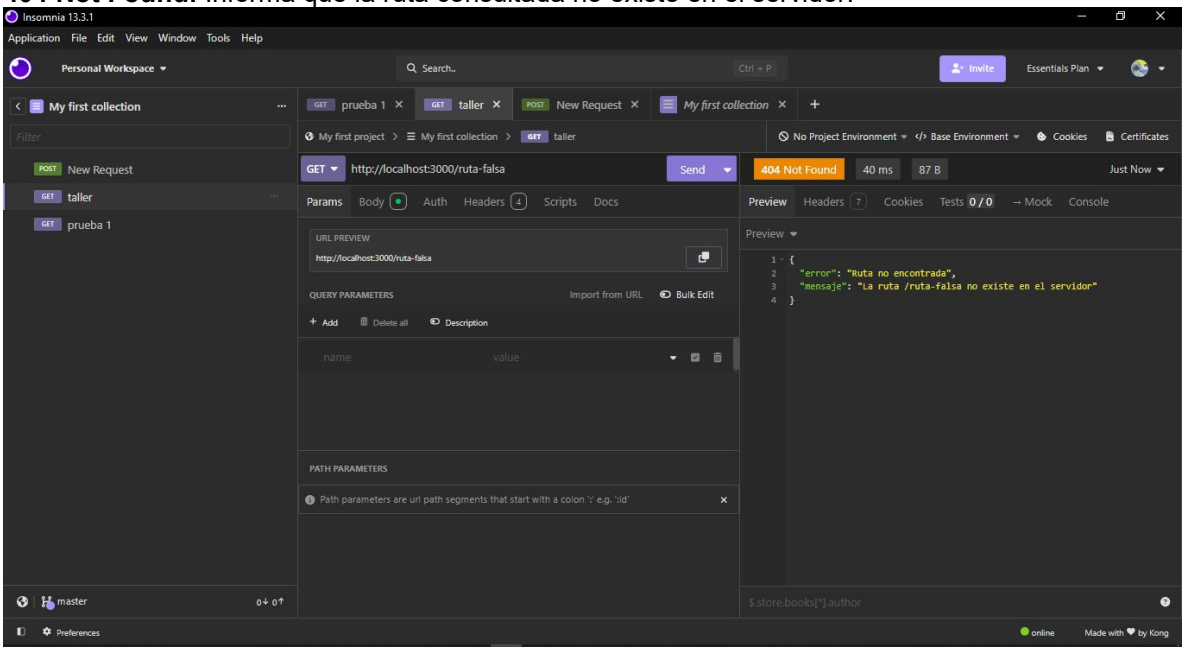


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

400 Bad Request: Avisa que el cliente envió datos incompletos o inválidos (como faltar un campo o un correo sin @).

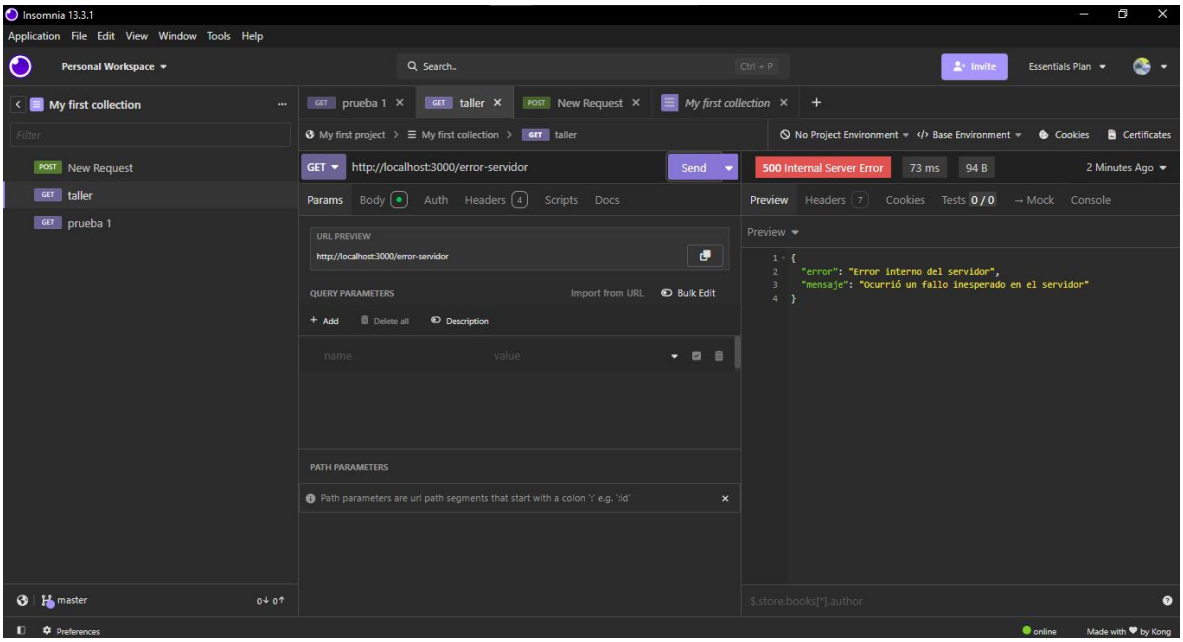


404 Not Found: Informa que la ruta consultada no existe en el servidor.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

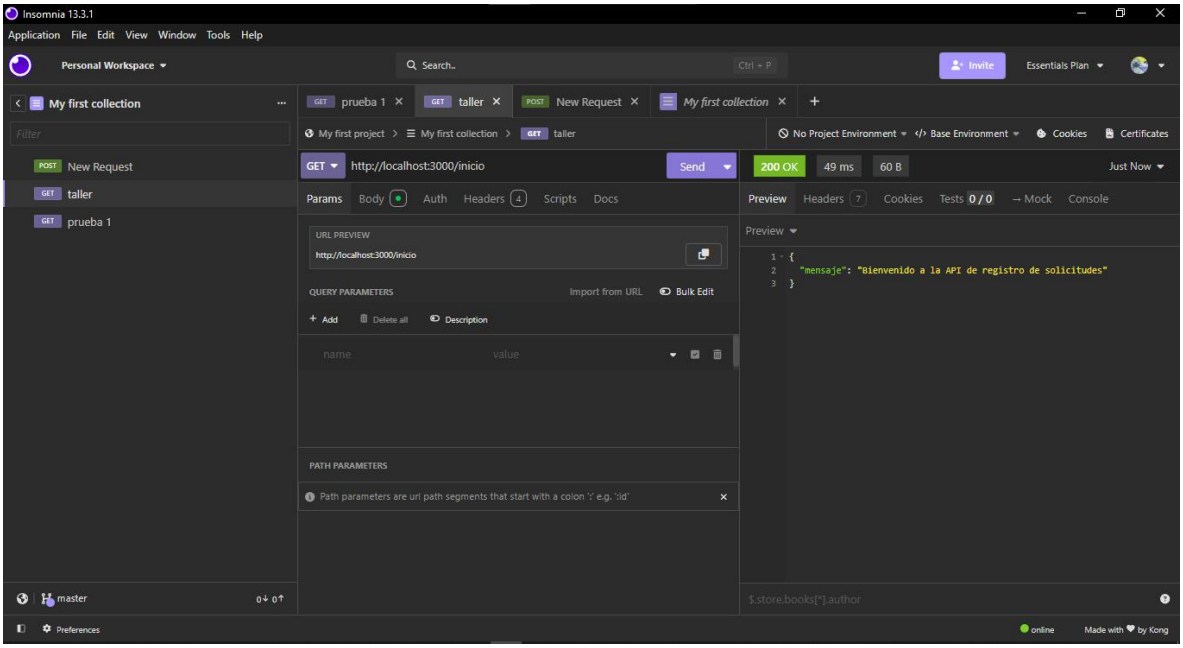
500 Internal Server Error: Indica que ocurrió un fallo inesperado del lado del servidor debido a un error interno no controlado.



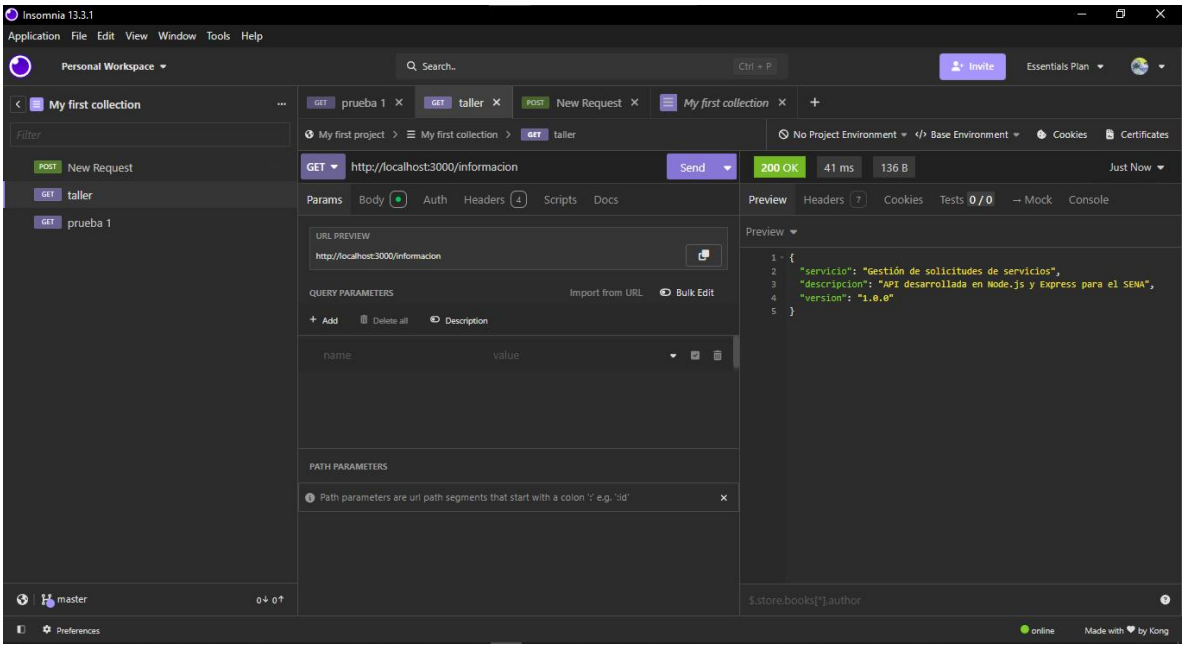
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Punto 4:Pruebas de la API

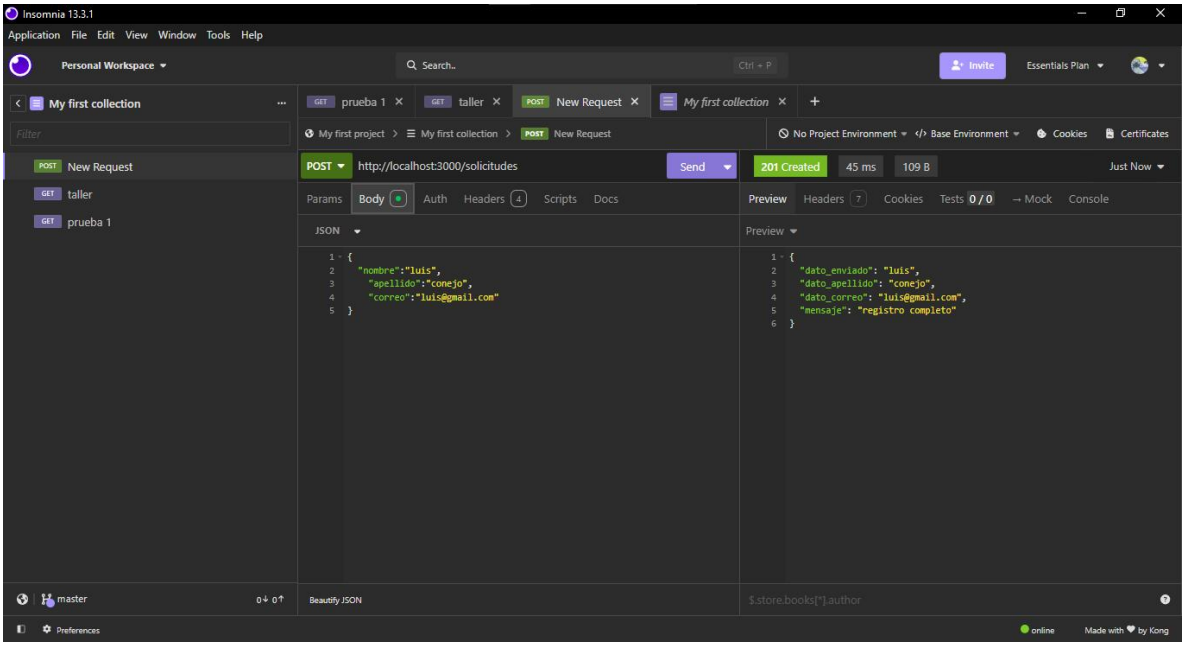
- GET /inicio: verificar que devuelve una respuesta JSON.



- GET /informacion: verificar que devuelve la información general.



- POST /solicitudes con nombre, apellido y correo válidos.



- Probar qué ocurre si se envía un cuerpo JSON y explicar el papel de express.json().

Prueba y papel de express.json(): Al mandar el JSON desde Insomnia en la petición POST, la API procesa los datos sin lío gracias a que express.json() hace de traductor, convirtiendo ese texto en un objeto de JavaScript fácil de usar (req.body). Si no pusieras esa línea, el req.body llegaría vacío (undefined) y la API ni se enteraría de lo que le enviaste.

Punto 5: Investigación sobre Middleware en Express

1. ¿Qué es un middleware en Express y cuándo se ejecuta?

Un middleware es como un peaje o un filtro por el que tiene que pasar la petición antes de llegar a su destino final. Se ejecuta justo en el medio del camino: cuando el cliente ya mandó la solicitud, pero antes de que el servidor le responda.

2. ¿Qué significan req y res?

req (Request / Petición): Es la bolsa donde viene todo lo que el cliente te envió (datos del formulario, el cuerpo en JSON, la URL, etc.).

res (Response / Respuesta): Es la herramienta que usa el servidor para responderle al cliente, ya sea mandándole un estado como 200 o 400, un mensaje o una lista de datos.

3. ¿Diferencia entre middleware global, de ruta específica y de manejo de errores?

Global: Aplica para absolutamente todas las rutas de la API (por ejemplo, el logger para ver qué entra).

Ruta específica: Solo actúa en una ruta concreta (por ejemplo, un filtro que solo revise credenciales cuando intentan entrar a /admin).

Manejo de errores: Es la red de seguridad que atrapa las fallas cuando algo se rompe en el código. Recibe 4 parámetros: (err, req, res, next).

4. ¿Qué hace express.json() y qué pasa si no se pone?

¿Qué hace?: Coge el texto plano en JSON que viene del cliente y lo traduce a un objeto de JavaScript entendible.**¿Qué pasa si no está?:** El servidor no entiende qué le mandaron y req.body llega como undefined, así que la API no puede leer ningún dato enviado.

5. ¿Por qué importa el orden de los middleware y las rutas?

Porque Express lee el código en orden, de arriba a abajo. Si pones express.json() o las validaciones después de las rutas, la petición llega a la ruta sin ser traducida ni revisada. Y si pones el middleware del 404 al principio, te va a rebotar todo antes de llegar a las rutas reales.

6. ¿Qué pasa si un middleware no responde ni deja continuar el flujo?

La petición se queda "colgada" en el aire. El cliente (como Insomnia) se queda esperando indefinidamente hasta que salte un error por tiempo de espera (*timeout*).

7. ¿Diferencia entre un middleware y la función de la ruta?

Middleware: Procesa, revisa o transforma la petición y le pasa el turno a la siguiente función usando next().

Función de la ruta: Es la parada final. Hace la lógica de negocio y cierra el ciclo respondiendo al cliente con un res.json().

8. ¿Qué riesgos hay si se omiten las validaciones?

Datos basura: Se pueden guardar campos vacíos o correos con formatos raros.

Caídas del servidor: La API intenta leer cosas que no existen y saca errores 500.

Seguridad: Cualquiera podría colarse a rutas protegidas o alterar información sin permiso.

9. Ejemplo de la vida real (sin código)

Imagina que vas a entrar a una **discoteca**:

Middleware global: El tipo de la entrada le pide el documento a todo el mundo sin excepción para dejarlo pasar al sitio.

Middleware de ruta específica: El filtro de la zona VIP, que solo revisa la manilla de los que intentan pasar a ese espacio reservado.

Función de la ruta: El barman que te atiende y te entrega el trago (la respuesta final).

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Punto 6: Documento técnico obligatorio: paso a paso

paso o decisión	**Por qué se realiza (Justificación)**	**Qué ocurriría si se omite**
Instalación de Node.js y npm	Aporte: Node.js nos da el entorno para ejecutar JavaScript en el servidor, y npm es el gestor que nos permite descargar e instalar librerías. Comando ejecutado: Instalar el ejecutable oficial de Node.js.	Sin Node.js no habría cómo interpretar ni correr el código backend en la máquina. Sin npm no podríamos instalar herramientas clave como Express.
Creación de la carpeta y apertura del proyecto	**Organización:** Permite guardar y aislar todos los archivos y dependencias del proyecto en un único espacio de trabajo dentro de VS Code.	Los archivos quedarían regados por todo el disco duro, dificultando las rutas de acceso, la instalación de paquetes y el control de versiones con Git.
Inicialización del proyecto (package.json)	**Administración:** Crea el archivo package.json que registra la información del proyecto, las librerías instaladas y los scripts de arranque. **Comando ejecutado:** npm init -y	Sin este archivo, npm no sabría qué dependencias usa la app. Se perderían los scripts de inicio y sería imposible replicar el proyecto en otro computador.
Instalación de Express	**Facilidad de servidor:** Es el framework que simplifica la creación del servidor, el manejo de rutas y la gestión de peticiones HTTP. **Comando ejecutado:** npm install express	Habría que crear el servidor desde cero usando el módulo nativo http de Node.js, requiriendo cientos de líneas de código extra y muy difíciles de mantener.
Instalación y configuración de Nodemon	**Soporte al desarrollo:** Reinicia automáticamente el servidor en la consola cada vez que guardamos un cambio en el código. **Comando ejecutado:** npm	Tendrías que apagar y encender manualmente el servidor en la terminal (Ctrl + C y luego node index.js) tras cada mínimo cambio que hagas en el código.

	install --save-dev nodemon	
Archivo principal y puerto	**Punto de entrada:** El archivo principal (index.js o app.js) arranca Express, y el puerto (3000) define la puerta de red por donde el servidor escucha. **Configuración en código:** app.listen(3000)	Si el servidor no escucha en un puerto activo, cualquier cliente (como Insomnia o el navegador) recibirá un error de conexión rehusada (ECONNREFUSED).
Middleware para analizar JSON (express.json())	**Preparación de datos:** Convierte el texto plano en formato JSON que envía el cliente en un objeto de JavaScript manipulable (req.body). **Configuración en código:** app.use(express.json())	Al intentar leer los datos en peticiones POST, req.body llegará como undefined y la API no podrá leer la información enviada por el cliente.
Middleware propio	**Monitoreo:** Muestra en la consola cada petición que llega (método HTTP y ruta) en tiempo real para facilitar la depuración. **Configuración en código:** app.use((req, res, next) => ...)	No tendrías visibilidad en la terminal sobre qué peticiones están entrando. Y si se coloca en el orden incorrecto o falta next(), se bloquearía el flujo.
Rutas GET y POST	**Propósito:** GET se usa para solicitar o consultar información del servidor, y POST para enviar y registrar datos nuevos desde el cliente. **Configuración en código:** app.get(), app.post()	Si falta una ruta o su verbo HTTP, el servidor responderá con un error 404 Not Found, dejando esa función inhabilitada para el usuario.
Validación y códigos HTTP	**Comunicación estandarizada:** Informan con claridad al cliente sobre el	Responder siempre con éxito (200 OK) ocultaría errores reales, confundiendo al cliente

	resultado usando códigos estándar (200, 201, 400, 404, 500). **Configuración en código:** res.status(400).json(...)	frontend y permitiendo el ingreso de datos basura a la app.
Pruebas de la API	**Demostración:** Permite comprobar en Insomnia que la API responde bien tanto con datos correctos como con errores o campos faltantes. **Ejecución:** Pruebas desde Insomnia.	Pasarían inadvertidos fallos de validación, rutas mal escritas o errores del servidor que harían colapsar la aplicación en producción.
Git y repositorio remoto	**Seguimiento y respaldo:** Permite guardar el historial de cambios, trabajar organizadamente y respaldar el código de forma segura en la nube (GitHub). **Comandos ejecutados:** git init, git add ., git commit -m "...", git push	Si le pasa algo a tu computador, perderías todo el trabajo. Además, no habría forma de auditar versiones anteriores o revertir errores en el código.